DAY 17

Object-Oriented Design and UML Diagrams

Software Engineering & Project Management • Student Learning Book

Learning outcomes

• Explain the purpose of object-oriented design and its relationship to requirements and architecture.

• Identify classes, objects, attributes, operations, and relationships.

• Read and sketch use-case, class, sequence, activity, state-machine, and deployment diagrams.

• Choose an appropriate UML diagram for a design question.

• Apply UML to a College Event Management System (CEMS).

Running case study: CEMS allows students to browse events and register; organizers create events, review registrations, and record attendance.

1. From Requirements to Design

Requirements describe what stakeholders need the system to do and the constraints it must satisfy. Design turns those requirements into a solution: the major parts, their responsibilities, their data, and how they collaborate.

Example requirement: “A student can register for an event only when registration is open and capacity remains.” Design questions include: Which class owns the registration rule? What information must be stored? Which objects interact when the student submits the form?

UML (Unified Modeling Language) is a standardized visual language for describing software structure and behavior. A diagram is a model of selected aspects of a system—not a replacement for clear requirements or implementation.

2. Object-Oriented Design Essentials

2.1 Class and object

A **class** describes a category of objects, including data and behavior. An **object** is a particular instance of a class. For example, `Event` is a class; “Tech Fest 2026” is one event object.

2.2 Attributes and operations

Attributes describe information held by an object, such as `eventId`, `title`, and `capacity`. Operations describe behavior, such as `publish()` or `hasCapacity()`.

2.3 Encapsulation and responsibility

Encapsulation keeps related state and behavior together and controls how internal state is changed. A Registration object or service should not allow arbitrary updates that bypass the capacity and duplicate-registration rules.

2.4 Abstraction

Abstraction exposes the important behavior while hiding unnecessary implementation details. A user clicks “Register”; they do not need to know how the database transaction is implemented.

2.5 Relationships

Relationship	Meaning	CEMS example
Association	Objects are linked or communicate	Student registers for Event

Relationship	Meaning	CEMS example
Aggregation	A whole has parts that can exist independently	An organizing committee has members who exist independently
Composition	A strong whole–part relationship; part lifecycle depends on whole	A registration line item may belong exclusively to a particular registration record, depending on the domain model
Generalization / inheritance	A specialized type inherits common characteristics	Organizer and Student may share common user properties, if the domain rules justify this
Dependency	One element temporarily uses another	RegistrationService uses a notification interface

Use inheritance only when the “is-a” relationship is meaningful. Two classes sharing a few fields does not automatically mean one should inherit from the other.

3. UML Diagram Families

UML diagrams are commonly grouped into structural diagrams (what the system is made of) and behavioral diagrams (what the system does over time).

Question you need to answer	Useful diagram
Who uses the system and what goals do they have?	Use-case diagram
What classes, attributes, operations, and relationships exist?	Class diagram
In what order do objects exchange messages?	Sequence diagram
What workflow steps and decisions occur?	Activity diagram
How does one object change state in response to events?	State-machine diagram
What software runs on which hardware or execution nodes?	Deployment diagram

4. Use-Case Diagram

A use-case diagram shows external actors and the goals they achieve with the system. It provides a high-level view of system scope, not a detailed screen flow.

CEMS actors: Student, Organizer, and Notification Service. Example use cases: Browse Events, Register for Event, Cancel Registration, Create Event, View Attendees, Mark Attendance.

Text sketch:

```
Student → (Browse Events)
Student → (Register for Event)
Student → (Cancel Registration)
Organizer → (Create / Update Event)
Organizer → (View Attendees)
Organizer → (Mark Attendance)
System → Notification Service (Send Confirmation)
```

Include and extend

<<include>> represents required reused behavior. For example, “Register for Event” may include “Validate Registration”. **<<extend>>** represents optional or conditional behavior added under defined conditions. Use these relationships carefully; do not add them simply to make the diagram look complex.

Boundary: draw a system rectangle around the use cases. Actors remain outside it. Name the boundary with the system name.

5. Class Diagram

A class diagram shows static structure. A class box commonly has three compartments: class name, attributes, and operations.

Example sketch:

```
+---------------------------+
| Event |
+---------------------------+
| - eventId: String |
| - title: String |
| - capacity: Integer |
| - registrationOpen: Boolean|
+---------------------------+
| + hasCapacity(): Boolean |
| + publish(): void |
+---------------------------+
```

Visibility symbols commonly used in UML: + public, - private, # protected. Multiplicity describes how many instances may participate in a relationship.

Multiplicity	Meaning	Example
1	Exactly one	A registration belongs to one student
0..1	Zero or one	A registration may have zero or one cancellation timestamp
0..*	Zero or many	An event may have many registrations
1..*	One or many	A team contains at least one member

Possible CEMS classes: Student, Organizer, Event, Registration, AttendanceRecord, NotificationService. Domain rules should determine whether Organizer is a subtype of User or a role associated with a User.

6. Sequence Diagram

A sequence diagram explains how participants exchange messages in time order. Time usually moves from top to bottom; participants appear across the top and have lifelines.

Registration example:

```
Student → RegistrationPage: select event and submit
RegistrationPage → RegistrationController: register(eventId)
RegistrationController → RegistrationService: register(student, event)
RegistrationService → Event: hasCapacity()
RegistrationService → RegistrationRepository: save(registration)
RegistrationService → NotificationService: sendConfirmation()
RegistrationController → RegistrationPage: success / error response
```

The diagram should also show important alternatives when relevant: event full, registration closed, duplicate registration, or database failure. It is important that the actual design handles these outcomes safely.

7. Activity Diagram

An activity diagram models workflow, including actions, decisions, and possible parallel activities.

CEMS registration workflow:

```
(Start)
↓
Select event
↓
Submit registration
↓
Is registration open?
■■ No → Show 'Registration closed' → (End)
■■ Yes → Is capacity available?
■■ No → Show 'Event full' → (End)
■■ Yes → Save registration → Show confirmation → (End)
```

Decision branches should be labeled with conditions, such as [yes] and [no]. Use activity diagrams for workflows; use sequence diagrams when the main concern is which participant sends each message and in what order.

8. State-Machine Diagram

A state-machine diagram describes the lifecycle of one entity. It is useful when the same object behaves differently depending on its current state.

Example: Event lifecycle

```
Draft → Published → Registration Open → Registration Closed → Completed
■ Cancelled (where cancellation is permitted)
Transitions occur because of events or conditions, such as organizer publishing the event or the
registration deadline being reached.
```

Define permitted transitions. For example, a completed event should not normally return to Draft without an explicit business rule. State diagrams help expose missing or contradictory lifecycle rules.

9. Deployment Diagram

A deployment diagram shows the physical or execution environment and where software artefacts run.

CEMS deployment sketch:

```
Student / Organizer device
↓ HTTPS
Web server / frontend hosting
↓ HTTPS API call
Application server / backend API
↓ secured database connection
Database server
■ external email/SMS provider
```

Deployment diagrams help teams discuss network boundaries, hosting, security, availability, and operational responsibilities. They are different from class diagrams: a class diagram shows software types; a deployment diagram shows runtime nodes and deployment relationships.

10. How to Build UML from a Requirement

1. Read the requirement and identify the user goal.

2. Identify actors and system boundary; sketch a use-case diagram.

3. Extract important nouns as candidate classes and verbs as candidate operations or interactions.

4. Assign each class a clear responsibility; add attributes and relationships.

5. Choose one important scenario and draw its sequence diagram.

6. Draw an activity diagram for the workflow and decisions.

7. Model important lifecycle states where behavior depends on status.

8. Draw a deployment diagram if runtime infrastructure matters.

9. Review every diagram against the requirement and check that names and rules remain consistent.

> Do not blindly convert every noun into a class or every verb into a method. Confirm each candidate against the problem domain and the system's responsibilities.

11. Common Design Errors

• Using a use-case diagram as if it were a detailed process flow.

• Adding classes with no clear responsibility or creating one enormous “Manager” class.

• Forgetting multiplicity or leaving relationships ambiguous.

• Showing only the successful path in a sequence or activity diagram.

• Using inheritance where composition or association is more appropriate.

• Making diagrams visually attractive but inconsistent with the SRS.

• Drawing deployment nodes when the question is actually about class responsibilities.

12. Classroom Practical: CEMS UML Portfolio

Create a small UML portfolio for the College Event Management System.

1. Use-case diagram with at least two actors and six use cases.

2. Class diagram with at least five classes, attributes, operations, relationships, and multiplicities.

3. Sequence diagram for event registration, including one failure path.

4. Activity diagram for event registration with decisions.

5. State-machine diagram for the Event lifecycle.

6. Deployment diagram showing client, application server, database, and notification provider.

> **Suggested tools:** diagrams.net (Draw.io), PlantUML, or another UML editor. The tool is less important than correct notation, readable labels, and consistency.

13. Review Questions with Suggested Answers

1. What is UML?

Suggested answer: A standardized visual modeling language for representing software structure and behavior.

2. What is the difference between a class and an object?

Suggested answer: A class defines a type; an object is an instance of that type.

3. Which diagram shows actors and system goals?

Suggested answer: A use-case diagram.

4. Which diagram shows classes and their relationships?

Suggested answer: A class diagram.

5. When should you use a sequence diagram?

Suggested answer: When you need to show time-ordered messages exchanged by participants in a scenario.

6. How is an activity diagram different from a sequence diagram?

Suggested answer: An activity diagram emphasizes workflow and decisions; a sequence diagram emphasizes messages between participants over time.

7. What does 0..* mean?

Suggested answer: Zero or many instances may participate in the relationship.

8. What does a state-machine diagram model?

Suggested answer: The states of an entity and the events or conditions that cause transitions.

9. What does a deployment diagram show?

Suggested answer: Runtime nodes or infrastructure and where software artefacts are deployed.

10. Why should UML diagrams be checked against the SRS?

Suggested answer: To keep the design consistent with stakeholder requirements and avoid missing or contradictory behavior.

14. Glossary

Term	Meaning
Class	Definition of data and behavior for a category of objects
Object	An instance of a class
Attribute	Data associated with a class or object
Operation	Behavior provided by a class
Association	A relationship between model elements
Multiplicity	The allowed number of instances participating in a relationship
Actor	An external role that interacts with a system
Lifeline	A participant's presence over time in a sequence diagram
State	A condition or status in an entity's lifecycle
Deployment node	A runtime environment or device on which software executes

15. Homework

1. Choose a Library Management System and create a use-case diagram with at least three actors.

2. Create a class diagram for Student, Book, Loan, and Librarian. Add reasonable attributes, operations, relationships, and multiplicities.

3. Draw a sequence diagram for borrowing a book, including the case where no copy is available.

4. Draw an activity diagram for student event registration and a state-machine diagram for the event lifecycle.

5. Write five sentences explaining how your diagrams map back to requirements in the SRS.

Next chapter: Software Testing and Quality Assurance — verification, validation, test levels, and test design.